

Evolving the Ultimate Italian Plumber

André Silva and Tiago Lopes Gonçalves

Universidade de Coimbra

Abstract. This paper presents an evolutionary approach to designing autonomous controllers for the Super Mario environment. The selected method is Genetic Programming, where each individual represents an executable controller that maps local sensory observations to actions. The main objective is level traversal and completion, while enemy information is incorporated through perception and fitness shaping. A secondary combat-oriented task was also implemented, but the final evaluation focuses on the general completion objective. The evolved controllers are compared against a Random Search GP baseline over five independent seeds and three levels. Results show that the evolved GP substantially outperforms the random baseline in completion rate, maximum distance, and total fitness, although with significant variability across seeds.

Keywords: Evolutionary Algorithms · Genetic Programming · Super Mario · Game AI

1 Introduction

Video games provide a useful testbed for Artificial Intelligence because they combine reproducible environments with real-time decision making, partial observation, and sequential action planning. Platform games such as Super Mario are especially relevant because successful play requires an agent to coordinate basic actions, such as moving, jumping, and running, while reacting to obstacles, gaps, and enemies.

In this work, we address the evolution of an autonomous controller for the Super Mario environment using Genetic Programming (GP). Instead of manually designing a fixed policy, the objective is to evolve executable programs that directly control Mario based on the information provided by the environment.

The main goal of the project is to produce agents capable of progressing through and completing levels. Enemy interaction is also considered, both through enemy-aware sensory primitives and through a combat-oriented fitness function. However, the final experimental evaluation focuses primarily on the general completion objective, since reliable level traversal is the first required stage before optimizing more specific combat behavior.

2 Problem Statement

The Super Mario environment is a two-dimensional platformer in which the agent must move from left to right, avoid hazards, react to enemies, and reach

the end of the level. The agent interacts with the game through a discrete action vector:

[backward, forward, crouch, jump, speed/bombs]

Each position in the vector is binary, indicating whether the corresponding action is active or inactive at a given time step. The controller must therefore learn not only which individual actions are useful, but also which combinations of actions are appropriate in different states of the environment.

The environment provides several forms of sensory information, including Mario’s position, whether Mario is on the ground, whether he can jump, a local observation grid representing terrain, and enemy-related observations. In this project, the 22×22 local grid is used to detect terrain features, obstacles, gaps, and enemies in the region surrounding Mario.

3 Methodology

3.1 Evolutionary Strategy

Between the two possible approaches, Genetic Programming and Neuroevolution, we selected Genetic Programming with a **CodeAgent**. This choice was motivated by the interpretability of GP: the evolved controller is an executable Python-like program, allowing direct inspection of the decision logic learned by evolution.

Each individual in the population represents a symbolic program. During evaluation, this program is converted into a **corre** function, which receives sensory inputs and writes actions to the controller vector.

3.2 Genotype Representation

Individuals are represented as strongly typed GP trees using the DEAP framework. The primitive set contains control-flow constructs, action-setting operations, and sensory checks. The main primitives include:

- conditional structures: **if-then** and **if-then-else**;
- sequential execution of expressions;
- action assignment operations for the available controller buttons;
- enemy checks using the local enemy observation grid;
- landscape checks using the local terrain observation grid.

The action terminals include movement to the right and left, crouching, jumping, and speed/bombs. This ensures that the evolved programs can use the full controller action space. The sensory terminals include the basic state indicators **on_ground** and **can_jump**, together with local grid positions, comparison operators, enemy types, and landscape values.

3.3 Initialization

The initial population is generated randomly using a safe grow procedure. This procedure constructs typed GP trees while respecting the available primitives and terminals for each type. The population size used in the final experiments was 120 individuals.

3.4 Variation Operators

The evolutionary process uses both crossover and mutation. Crossover is implemented as one-point leaf-biased crossover, allowing subtrees to be exchanged between two parent programs. Mutation is implemented as a mixed strategy composed of:

- uniform mutation, replacing a subtree with a newly generated subtree;
- node replacement mutation, replacing individual nodes;
- shrink mutation, reducing tree size.

A static tree-height limit is applied to both crossover and mutation to control code bloat and avoid excessively large programs.

3.5 Selection Mechanism

Parent selection is performed through tournament selection with tournament size 2. A relatively small tournament size was chosen to reduce selection pressure and preserve diversity. Survivor selection uses elitism: the top 2 individuals are copied directly to the next generation, ensuring that the best solution found so far is not lost.

3.6 Development Decisions

Several changes were introduced during development to make the evolutionary process more suitable for the Mario environment.

The first decision was to expand the original controller vocabulary. A very small grammar based only on moving right and jumping would make the search space easier, but would also restrict the behaviors that evolution could discover. Therefore, the final grammar includes the full action space, including left, right, down, jump, and speed. This allows evolved programs to express more complex behaviors, such as running jumps, stopping, crouching, or temporarily moving backwards.

The second decision was to include local perception. Instead of evolving blind action sequences, the controller can inspect both the landscape and enemy grids around Mario. This allows the program to condition its behavior on nearby terrain or enemies. The controller is still reactive, because it only uses local observations, but this is more appropriate than a purely action-based policy.

```

Initialize population with typed safe grow
Evaluate all individuals
for generation = 1 to 200 do
    Preserve best 2 individuals
    Select parents using tournament selection
    Apply crossover and mixed mutation
    Inject random immigrants
    Increase mutation if stagnation is detected
    Re-evaluate invalid/new individuals
    Save best individual and fitness statistics
end for
Return best evolved controller

```

Fig. 1. Simplified pseudocode of the Genetic Programming procedure.

The third decision was to use a typed safe generation procedure. During early development, randomly generated trees could easily become invalid or structurally unsuitable. The safe grow procedure ensures that every generated node has compatible primitives or terminals, reducing invalid individuals and making the evolutionary process more stable.

The fourth decision was to add bloat control and diversity mechanisms. Genetic Programming can produce unnecessarily large programs or converge prematurely to simple local optima. For this reason, tree height was limited, tournament size was kept small, elitism was used to preserve the best individuals, and random immigrants were introduced to maintain exploration.

3.7 Diversity and Stagnation Control

Preliminary experiments showed that the population could become trapped in local optima. To address this, several diversity-preserving mechanisms were added:

- random immigrants are inserted every generation;
- mutation probability increases when the best fitness stagnates;
- population diversity is monitored using the ratio of unique individuals;
- partial restarts are triggered after prolonged stagnation.

These mechanisms are intended to increase exploration while preserving the best individuals through elitism. This is particularly important in GP, where a population can quickly converge to simple but suboptimal behaviors, such as repeatedly jumping or always moving right without reacting to terrain.

3.8 Fitness Design Rationale

The fitness function was the most important design component of the project. In the Mario environment, sparse rewards are difficult for evolution: if the agent is only rewarded for completing the level, most early individuals receive poor feedback and the search process stagnates. To address this, the fitness function was shaped with intermediate rewards that guide the agent toward useful behavior.

Two task formulations were developed. The first, **MoveForward**, targets the general completion objective. The second, **Hunter**, targets the combat objective. Although both were implemented, the final experimental evaluation prioritizes **MoveForward**, because reliable level traversal is the necessary first stage before optimizing enemy elimination.

MoveForward The **MoveForward** task was designed as the primary fitness function. Its objective is to evolve agents that can traverse and complete levels. This choice follows the staged nature of the assignment: before optimizing combat behavior, the controller must first learn to move reliably through the level.

The main reward signal is horizontal progress. Positive reward is assigned when Mario moves to the right, and additional reward is given when a new maximum horizontal position is reached. This encourages not only momentary movement, but also sustained progression through the level. Moving backwards is penalized, since it generally delays completion.

A large terminal bonus is assigned when the level is completed, and a penalty is applied when the agent dies or fails. The terminal reward is also combined with remaining time, encouraging faster completion. To avoid degenerate behavior, the fitness function penalizes inactivity when the agent remains stuck for several frames.

Action shaping was added to make the search more informative. The agent receives small rewards for pressing forward and speed, while failing to move forward is penalized. Jumping is only rewarded when danger is detected ahead, preventing the population from converging to constant jumping. Danger detection uses the local scene grid to identify obstacles, enemies, and possible gaps.

Although the objective is mainly traversal, enemy information is still included. Enemies are treated as part of the danger-ahead mechanism, and enemy kills are optionally rewarded when the environment exposes kill counters. Therefore, the final controller is not blind to enemies, but enemy elimination is not the dominant pressure in this fitness function.

Hunter The **Hunter** task was implemented as an alternative combat-oriented formulation. In this task, the reward for enemy kills is much larger than in **MoveForward**, while the reward for forward progress is smaller. This shifts the objective from level traversal to enemy interaction.

The task also rewards being near enemies and performing useful jumps when enemies are ahead. At the same time, it penalizes damage, death, unnecessary jumps, and lack of progress. This formulation was intended to support the second stage of the assignment, where combat optimization becomes the main goal.

However, the final experiments reported in this paper focus on **MoveForward**. This was a deliberate choice: the evolved controller first needs to learn robust traversal before combat optimization can be meaningfully evaluated. The **Hunter** task is therefore presented as an implemented extension and as a basis for future combat-focused experiments.

3.9 Controller Grammar Design

A key part of the project was the design of the GP grammar. Since the selected representation was a **CodeAgent**, each GP individual had to be translatable into valid executable code. For this reason, a strongly typed grammar was adopted. The use of types prevents invalid programs from being generated, for example by avoiding the use of an action where a condition is expected.

The grammar was organized around the following typed elements:

- **Expressions:** executable statements that modify the action vector or combine other statements.
- **Conditions:** Boolean expressions used inside control structures.
- **Keys:** controller buttons such as right, left, down, jump, and speed.
- **Booleans:** values used to activate or deactivate a controller key.
- **Positions:** local grid offsets around Mario.
- **Comparators:** equality and inequality checks.
- **Enemy and landscape types:** constants used to inspect the observation grids.

The primitive set includes three main groups of operations. First, control-flow primitives allow the evolution of conditional behavior through **if-then**, **if-then-else**, and sequential execution. Second, action primitives allow the evolved program to set individual elements of the controller vector. Third, sensory primitives allow the program to query the local enemy and landscape grids.

3.10 Baseline Construction

The baseline used in the final comparison was not a generic random action agent. Instead, it was a Random Search GP baseline using the same general representation as the evolved controller. This is important because both methods operate over program-like controllers for the **CodeAgent**. The difference is that the evolved GP uses selection, crossover, mutation, elitism, and multiple generations, while the baseline simply samples random GP individuals and keeps the best one found.

This makes the comparison fairer than comparing against a completely different controller type. If the evolved GP performs better, the improvement can be attributed to the evolutionary process rather than to a different action representation.

4 Experimental Setup

The final GP configuration was:

- population size: 120;
- number of generations: 200;
- crossover probability: 0.45;

- base mutation probability: 0.30;
- maximum adaptive mutation probability: 0.75;
- elitism: top 2 individuals preserved;
- random immigrant rate: 8%;
- stagnation immigrant rate: 25%.

The evolved controller was compared against a Random Search GP baseline. The baseline uses the same agent class and program representation, but does not use evolutionary selection across generations. Instead, random GP programs are generated and evaluated, and the best random candidate is retained.

Five independent seeds were used. For each seed, one evolved GP controller and one Random Search GP baseline were produced. Each resulting agent was then evaluated on three levels, giving 15 evaluation runs per method.

4.1 Metrics

The following metrics were recorded:

- **Completion Rate (%)**: percentage of evaluation episodes in which the level was completed;
- **Maximum Distance**: furthest horizontal position reached before death or timeout;
- **Enemies Killed**: number of enemy eliminations recorded by the environment;
- **Total Fitness**: final task fitness accumulated during the episode.

Enemy eliminations were kept in the table for completeness. However, in the final evaluation they remained zero for both methods, making this metric non-informative for distinguishing the controllers.

5 Results

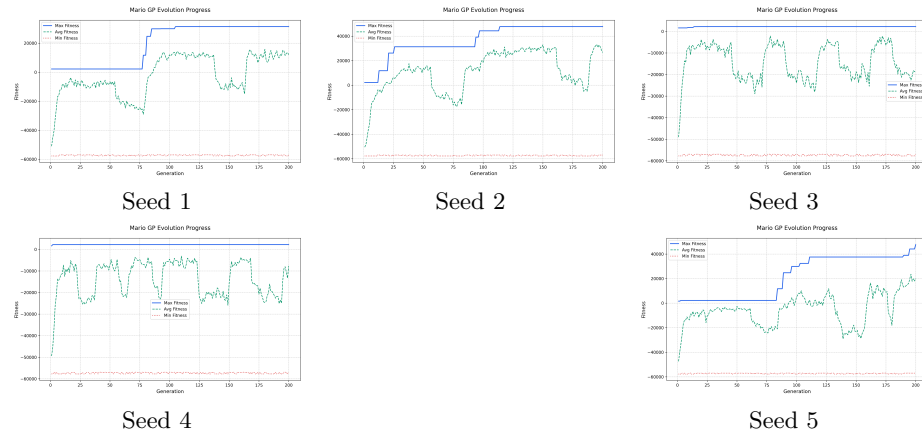
Table 1. Performance comparison between Random Search GP and Evolved GP over five seeds and three levels. Values are reported as mean $\pm$ standard deviation.

Metric	Random Search GP	Evolved GP	Improvement
Completion Rate (%)	0.00 $\pm$ 0.00	46.67 $\pm$ 51.64	+46.67 pp
Maximum Distance	314.05 $\pm$ 76.54	1390.66 $\pm$ 1024.78	+342.81%
Enemies Killed	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	N/A
Total Fitness	944.58 $\pm$ 485.34	10386.95 $\pm$ 8991.98	+999.64%

Table 2. Performance of the evolved GP controller for each seed and evaluation level.

Seed	Level	Completed	Max Distance	Enemies Killed	Fitness
1	0	Yes	2304.00	0	19080.96
1	1	No	2043.33	0	11570.60
1	2	No	490.62	0	2011.67
2	0	Yes	2304.00	0	19022.94
2	1	Yes	2400.00	0	19581.61
2	2	Yes	2368.00	0	19354.38
3	0	No	300.13	0	1419.73
3	1	No	210.94	0	291.79
3	2	No	210.94	0	292.67
4	0	No	300.13	0	1415.15
4	1	No	428.13	0	2177.94
4	2	No	427.60	0	1712.29
5	0	Yes	2304.00	0	19021.68
5	1	Yes	2400.00	0	19513.98
5	2	Yes	2368.00	0	19336.88

Table 2 presents the detailed performance of the evolved GP controller across the five independent seeds and the three evaluation levels. The results show strong variability between seeds. Seeds 2 and 5 completed all three levels, seed 1 completed only the first level, and seeds 3 and 4 failed to complete any level. This confirms that GP can discover successful traversal strategies, but also that the evolutionary process remains sensitive to stochastic initialization and search dynamics.

**Fig. 2.** Fitness convergence curves for the five independent GP runs. Each plot shows the evolution of minimum, average, and maximum fitness across generations.

The results show a clear advantage of the evolved GP controller over the Random Search GP baseline. The random baseline never completed a level, obtaining a completion rate of 0.00%. In contrast, the evolved GP achieved an average completion rate of 46.67%. This corresponds to 7 completed levels out of 15 evaluation runs.

The maximum distance metric also shows substantial improvement. The evolved GP reached an average maximum distance of 1390.66, while the random baseline reached only 314.05. This represents an improvement of approximately

342.81%. The same tendency is visible in total fitness, where the evolved GP obtained an average of 10386.95, compared to 944.58 for the random baseline.

5.1 Seed Variability

The standard deviation of the evolved GP results is high, particularly for completion rate, maximum distance, and total fitness. This indicates substantial variability between seeds. Some seeds discovered strong policies capable of completing all three tested levels, while others remained trapped in weaker behaviors and failed to complete any level.

This variability is expected in Genetic Programming. The evolutionary process depends on stochastic initialization, crossover, mutation, selection, and random immigrant insertion. A useful combination of actions and conditions may appear early in one seed and much later, or not at all, in another seed. This observation reinforces the importance of reporting averages and standard deviations over multiple seeds rather than relying on a single run.

5.2 Enemy Elimination Metric

The enemies killed metric remained zero for both methods. This does not mean that enemy information was absent from the controller design. The GP grammar includes enemy-related sensory checks, and the `MoveForward` fitness function uses enemies as part of its danger-ahead detection. Additionally, the `Hunter` task was implemented as a specific combat-oriented objective.

However, the final experiments prioritized the general completion objective. Since the selected evaluation task was focused on traversal, and because the environment did not expose reliable kill counters during the final runs, enemy eliminations were not informative in the final comparison. For this reason, completion rate, maximum distance, and total fitness are the main indicators used to evaluate performance.

6 Discussion

The evolved GP controller outperformed the Random Search GP baseline across the main evaluation metrics. This indicates that the evolutionary process was able to exploit selection, crossover, mutation, and elitism to discover better controllers than random program generation alone.

The behavior of the random baseline was limited. Randomly generated controllers often failed to maintain consistent forward movement, reacted poorly to obstacles, and did not produce reliable action combinations. As a result, the random baseline remained far from level completion in all tested cases.

The evolved controllers learned more useful strategies, particularly sustained forward movement combined with jumping and speed usage. The best evolved agents were able to complete all three tested levels, showing that the GP process

can discover effective control programs. Nevertheless, weaker seeds demonstrate that the search process remains unstable.

The diversity and stagnation mechanisms played an important role. During evolution, some runs showed long periods of stagnation followed by sudden improvements in fitness. This suggests that adaptive mutation and random immigrants helped the population escape local optima and discover better behavior.

Despite these positive results, several limitations remain. First, performance is highly seed-dependent. Second, the controller remains reactive and local, relying mostly on the current observation grid rather than long-term planning. Third, the final evaluation did not demonstrate combat optimization, since enemy eliminations were not observed in the collected runs. Finally, only one episode per level was used in the final comparison, so additional episodes would provide more stable estimates.

7 Conclusion

This work demonstrated that Genetic Programming can evolve autonomous Super Mario controllers that substantially outperform a Random Search GP baseline. The evolved GP achieved higher completion rate, greater maximum distance, and much higher total fitness than the random baseline over five seeds and three levels.

The results support the use of evolutionary algorithms for generating interpretable game-playing agents. At the same time, the high standard deviation shows that GP remains sensitive to initialization and stochastic variation. Future work should focus on increasing robustness across seeds, evaluating more episodes per level, improving the combat-oriented objective, and combining traversal and enemy-elimination rewards in a multi-objective formulation.

Statement on the Use of Artificial Intelligence. Artificial intelligence tools were used as support during the preparation of this work, including language revision, report structuring, debugging assistance, and the generation of some code fragments. All AI-assisted code was reviewed, adapted, and integrated by the authors. The design of the evolutionary approach, the experimental setup, the execution of the experiments, the reported results, and the final interpretation remained the responsibility of the authors. No experimental results, metrics, or conclusions were generated or fabricated by AI.

References

1. Togelius, J., Karakovskiy, S., Koutnik, J., Schmidhuber, J.: Super Mario Evolution. In: Proceedings of the IEEE Symposium on Computational Intelligence and Games, pp. 156–161 (2009)
2. Fortin, F.-A., De Rainville, F.-M., Gardner, M.-A., Parizeau, M., Gagné, C.: DEAP: Evolutionary Algorithms Made Easy. *Journal of Machine Learning Research* 13, 2171–2175 (2012)